

RSO-SLAM: A Robust Semantic Visual SLAM With Optical Flow in Complex Dynamic Environments

Liang Qin^{ID}, Chang Wu^{ID}, *Senior Member, IEEE*, Zhenyu Chen^{ID}, Xiaotong Kong^{ID}, Zejie Lv^{ID}, and Zhiqi Zhao^{ID}

Abstract—Visual Simultaneous Localization and Mapping (VSLAM) has undergone gradual development and found widespread application. However, existing VSLAM systems predominantly rely on static environment assumptions, leading to diminished robustness and localization accuracy in the presence of dynamic elements. Previous research has primarily employed geometric and semantic constraints to address dynamic regions of the scene. Nevertheless, their efficacy is limited in complex dynamic scenarios involving non-rigid objects, non-predefined motion targets, and low dynamic motion targets. Furthermore, the majority of dynamic SLAM methods are predominantly designed for indoor RGBD environments, resulting in a lack of generalizability. In this paper, a dynamic SLAM method that combines instance segmentation and optical flow called RSO-SLAM is proposed. RSO-SLAM is designed to operate effectively in diverse complex motion scenarios, both indoors and outdoors, and supports various visual sensor modes, including monocular, stereo, and RGBD setups. The proposed approach amalgamates semantic information and optical flow data by employing a “KMC:k-means + connectivity” based algorithm for motion region detection within the scene. Furthermore, it integrates an optical flow attenuation propagation strategy to facilitate meticulous motion probability computations and inter-frame propagation within each identified region. Our methodology’s superiority over existing dynamic SLAM approaches is firmly established through comprehensive evaluations across a diverse range of intricate dynamic scenarios. These evaluations encompass various conditions of high and low dynamism in both indoor and outdoor environments, accompanied by rigorous ablation experiments and real-world assessments. RSO-SLAM exhibits enhanced robustness and higher localization accuracy, rendering it well-suited for nearly all dynamic environments.

Index Terms—VSLAM, dynamic environments, instance segmentation, optical flow, dynamic region detection, motion probability.

I. INTRODUCTION

SIMULTANEOUS localization and mapping (SLAM) have emerged as a crucial technology for autonomous robots,

finding extensive applications in numerous unmanned devices. Within this domain, visual SLAM has garnered significant attention in the fields of computer vision and robotics, due to the camera’s attributes of high precision and low cost. Consequently, several high-performance VSLAM systems have been developed, including DSO [1], LSD-SLAM [2], and ORB-SLAM2 [3].

Despite the satisfactory performance achieved by current VSLAM systems in various scenarios, certain challenges remain unresolved. These systems are designed under the assumption of static environments, rendering them vulnerable to erroneous observations resulting from dynamic factors. Consequently, their accuracy and robustness are compromised. While the Random Sample Consensus (RANSAC) algorithm provides a mechanism to handle outliers and mitigate the impact of minor dynamics, its effectiveness diminishes when faced with a substantial number of moving objects within the scene.

To address the limitations and improve the robustness and accuracy of SLAM algorithms in dynamic environments, researchers have proposed diverse SLAM methodologies explicitly tailored to dynamic scenarios. These methodologies can be broadly classified into main groups: geometric-based [4], [5], [6], [7], [8], [9], [10] and semantic-based methods [11], [12], [13], [14], [15], [16], [17], [18], [19]. Furthermore, some SLAM methods [20], [21], [22], [23], [24], [25] incorporate optical flow to address dynamic scenes.

Geometric-based methods rely on geometric constraints to detect dynamic objects within the scene. However, they often overlook the potential motion exhibited by objects, leading to missed detection of moving targets. Conversely, semantic-based methods employ object detection or semantic segmentation techniques to identify and remove dynamic objects. Nevertheless, these methods often struggle to effectively detect unforeseen dynamic objects and exhibit limitations in accurately assessing the actual motion of predefined dynamic objects. For optical-flow-based methods, excessive reliance on optical flow can lead to decreased robustness. Furthermore, fusion strategies based on optical flow are often constrained and exhibit poor performance in detecting dynamic objects. Some researchers [26], [27] have attempted to integrate semantic-based and geometric-based approaches to handle the removal of moving objects. However, these combined methods tend to consume significant computing resources and are limited in their applicability to specific scenes.

Manuscript received 27 September 2023; revised 3 February 2024, 1 April 2024, and 10 May 2024; accepted 13 May 2024. Date of publication 30 May 2024; date of current version 4 October 2024. This work was supported by the Science and Technology Project Fund of Sichuan Province, China, under Grant 2018sz0364. The Associate Editor for this article was S.-H. Kong. (Liang Qin and Chang Wu are co-first authors.) (Corresponding author: Chang Wu.)

The authors are with the Institute of Information and Communication Engineering, University of Electronic Science and Technology of China (UESTC), Chengdu 611731, China (e-mail: qinliang6218@foxmail.com; changwu@uestc.edu.cn; m13408933791@163.com; m13086663208@163.com; 202222010904@std.uestc.edu.cn; 202252012118@std.uestc.edu.cn).

Digital Object Identifier 10.1109/TITS.2024.3402241

Overall, the current dynamic SLAM solutions exhibit several shortcomings:

- 1) In low-dynamic scenes, existing dynamic SLAM systems tend to incorrectly classify all moving objects as high-motion targets, leading to a substantial number of false removals and a significant reduction in the available image features.
- 2) When non-rigid objects, such as humans, are present in the scene and exhibit local motion, the prevailing dynamic SLAM methods often opt to entirely remove these objects, thereby diminishing the pool of useful image features and resulting in a decline in the accuracy of localization.
- 3) Semantic-based dynamic SLAM systems, which rely on predefined motion targets, frequently encounter challenges in effectively detecting and tracking moving objects that were not anticipated beforehand. As a consequence, the robustness of the system suffers severe deterioration.
- 4) The latest advancements in dynamic SLAM primarily concentrate on a single visual sensor mode, typically RGBD mode, which is suitable for indoor environments. However, when the scene undergoes changes or a different visual sensor mode is employed, these approaches lose their efficacy and exhibit limited adaptability.

To address the aforementioned challenges, we present RSO-SLAM for a dynamic environment that integrates instance segmentation and optical flow techniques. This is a high-performance, efficient, and robust monocular, binocular, RGB-D SLAM system suitable for dynamic region detection and motion probability calculation in indoor and outdoor scenes.

In this study, we enhance the ORB-SLAM2 method [3] by incorporating YOLOv5-seg [28] for instance segmentation and LiteFlowNet2 [29] for optical flow estimation. We also perform coarse matching to capture rigid flow in the scene. Scene segmentation is achieved using a “KMC:K-means + Connectivity” algorithm, and the integration of motion flow and semantic detection allows effective identification of moving regions. Furthermore, we utilize an “Optical Flow Attenuation Propagation” strategy, which involves combining magnitude and angle maps of motion flow. This approach is employed to address missed detections in motion regions and calculate corresponding motion probabilities.

The main contributions of this paper are as follows:

- 1) A motion region detection method based on the KMC algorithm is proposed, which combines semantic detection results to effectively detect all potential motion regions in the scene.
- 2) A motion probability calculation method based on the “optical flow attenuation propagation” strategy is introduced for compensating missed detection, calculating motion probabilities, and performing inter-frame propagation.
- 3) RSO-SLAM is evaluated on publicly available datasets, including TUM [30], BONN [31], and KITTI [32], in monocular, binocular, and RGB-D modes. Ablation

experiments and real-world experiments were also conducted.

The rest of this paper is organized as follows. Section II furnishes a comprehensive review of pertinent literature. In Section III, we delineate the overarching framework of the system. Sections IV and V expound upon the specific architecture of the RSO-SLAM system, encompassing the motion region detection algorithm and motion probability calculation method. The experimental results obtained from performing various experiments in the RSO-SLAM system are presented in Section VI. Finally, Section VII concludes the paper by summarizing the key findings and discussing current system limitations and potential future research directions.

II. RELATED WORK

A. Geometry-Based Dynamic SLAM

In scenarios lacking prior semantic information, utilizing geometric techniques to establish precise feature correspondences proves to be a crucial approach for tackling dynamic SLAM challenges.

Kim and Kim [4] proposed a dense visual odometry technique based on a background model, estimating a non-parametric background model from depth scene estimation. Li and Lee [5] introduced an RGB-D SLAM method that relies on frame-to-keyframe edges and incorporates static weights of foreground edge points into the Intensity-Aided Iterative Closest Point (IAICP) algorithm for pose estimation of the current frame. Sun et al. [6] approximated the detection of motion objects by using self-motion compensated image disparities and enhanced motion detection through particle filter-based motion tracking. They further proposed an online motion removal method based on RGB-D [7], involving the construction and incremental updating of a foreground model, along with the utilization of optical flow for tracking. Staticfusion [8] addressed the joint estimation of camera motion and the probabilistic segmentation of static/dynamic content within current RGB-D image pairs. Dai et al. [9] introduced the use of Delaunay triangulation to discern dynamic and static map points by examining changes in the edges of the triangle between consecutive frames. MVS-SLAM [10] using geometric constraints to refine localization and mitigate the impact of moving objects.

Geometric-based methodologies offer partial solutions to dynamic SLAM challenges, often exhibiting faster processing speeds. However, they are limited in their ability to leverage prior scene knowledge for mobile entity detection, and their robustness in dynamic scenes generally falls short of that demonstrated by semantic-based SLAM approaches.

B. Semantic-Based Dynamic SLAM

During the SLAM process, semantic-based dynamic SLAM methods rely on deep learning networks to perform tasks such as semantic segmentation or object detection, thereby providing reliable constraints.

Detect-SLAM [11] aims to identify objects in keyframes and continuously propagate the motion probability of keypoints in real time to mitigate the influence of dynamic

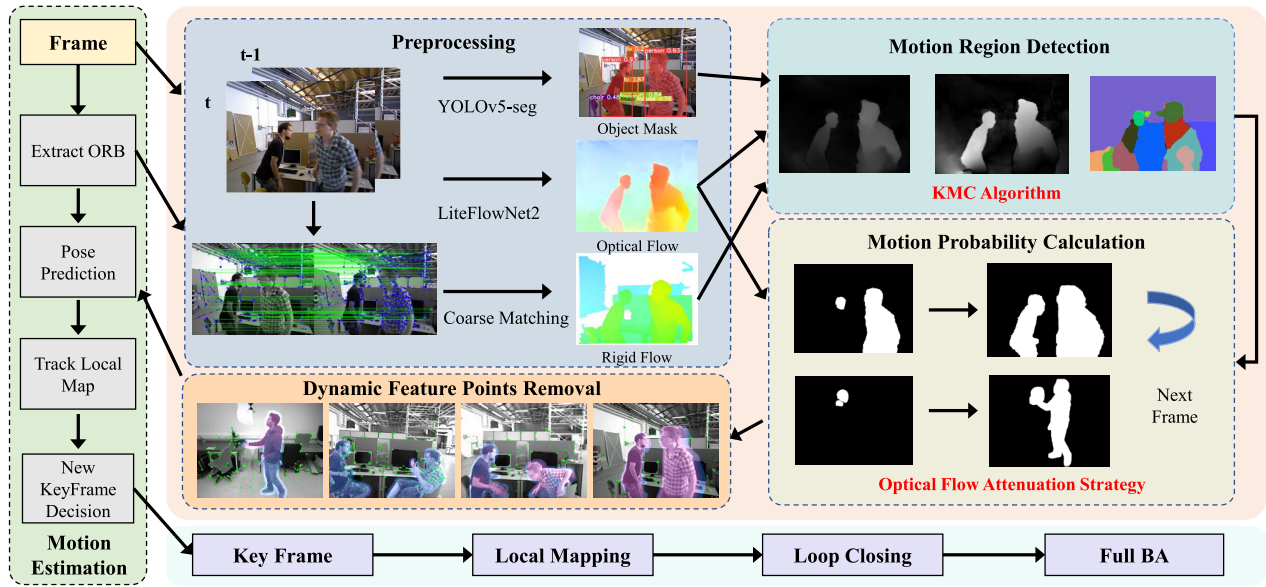


Fig. 1. System framework. RSO-SLAM is built upon the ORB-SLAM2 [3] framework and achieves precise removal of dynamic feature points through the Motion Region Detection and Motion Probability stages.

objects on SLAM. DS-SLAM [26] builds upon ORB-SLAM2 and SegNet [33] and utilizes epipolar constraints to assess the motion consistency of predefined semantic segmentation targets, including humans. Dyna-SLAM [27] combines MASK R-CNN [34] and multiview geometry techniques to effectively handle moving objects. Vincent et al. [12] perform semantic segmentation on object instances in images and employ an extended Kalman Filter (EKF) to identify, track, and eliminate dynamic objects from the scene. DP-SLAM [13] integrates geometric constraints with semantic segmentation results within a Bayesian probability estimation framework to track dynamic keypoints. Ji et al. [14] focus on semantic segmentation solely in keyframes, and cluster depth maps to identify moving objects, and employ reprojection error to remove known and unknown dynamic objects. Blitz-SLAM [35] uses masked depth information based on BlitzNet [15] and uses epipolar constraints to classify static and dynamic matching points within potential dynamic regions. Hu et al. [16] propose a visual SLAM algorithm for dynamic scenes based on object detection and a coarse-to-fine approach using static probabilities. Yan et al. [17] present a dynamic object detection module employing a polynomial residual model, integrating motion residual information from neighboring frames with potential motion data from the semantic segmentation module to facilitate motion segmentation within the scene. SC_LPR [19] enhances semantic SLAM in dynamic settings by integrating LSI-Net and SPT-Net for robust place recognition and employing LSI-Dataset for validation.

Semantic-based dynamic SLAM demonstrates efficacy in scenarios characterized by the motion of common objects. However, in the presence of unforeseen moving objects in the scene, conventional semantic-based methods for moving object removal tend to be ineffectual. Additionally, semantic-based SLAM often contends with challenges related to

over-segmentation, thereby exerting a detrimental impact on SLAM accuracy.

C. Optical-Flow-Based Dynamic SLAM

The deployment of optical flow to capture object motion has been a recurrent theme in dynamic SLAM methodologies. Classic visual odometry methods based on optical flow residuals, such as DSO [1] and VINS [36], often implicitly mitigate the influence of dynamic objects during the optimization process. However, this implicit handling may lead to estimation biases or error accumulation in dynamic environments. Hence, many Optical-Flow-Based methods opt for the explicit process of dynamic objects.

VDO-SLAM [20] maximizes tracking points in moving objects by concurrently computing and optimizing camera poses, static and dynamic points, and motion object poses using dense optical flow. FlowFusion [21] innovatively utilizes optical flow residuals for enhanced dynamic semantics in RGB-D point clouds, facilitating accurate and efficient dynamic/static segmentation in dense RGB-D SLAM for superior performance in both dynamic and static environments. DM-SLAM [22] combines optical flow, Mask R-CNN, and epipolar constraint for outlier judgment. STDyn-SLAM [23] integrates convolutional neural networks, optical flow, and depth maps to facilitate object detection in dynamic environments. Chen et al. [24] introduce a monocular SLAM algorithm for dynamic environments, employing semantic information and optical-flow-based geometric methods to filter dynamic feature points, demonstrating enhanced pose estimation accuracy. DYS-SLAM [25] improves dynamic SLAM with M-LK, an optical flow technique that efficiently identifies dynamic features by refining motion consistency. While optical flow has found widespread application in dynamic SLAM, current methods for its utilization remain limited. In RSO-SLAM,

we have expanded the scope of optical flow applications, introducing more diverse approaches.

III. SYSTEM OVERVIEW

The proposed RSO-SLAM system, which combines optical flow and instance segmentation, is implemented based on the framework of ORB-SLAM2 [3]. The specific framework is illustrated in Fig. 1.

The RSO-SLAM architecture comprises a primary thread like ORB-SLAM2 and two subordinate threads. When the primary thread engages in feature point detection, the two subordinate threads independently carry out optical flow calculation and instance segmentation. Following the completion of these three processes, the computation outcomes from the subordinate threads are transmitted to the main thread for motion area detection, motion probability calculation, and subsequent SLAM processes. Specifically, upon the introduction of a new camera frame to the system, the grayscale image is processed by the main thread for feature detection, while the RGB image is directed to the two subordinate threads for conducting instance segmentation and optical flow network inference. After the feature point detection phase, the system performs coarse matching between feature points of consecutive frames to establish initial correspondences. The obtained masks representing potential moving objects, dense optical flow maps, rigid flow, and motion region masks propagated from the previous frame are then input into the motion detection module.

Utilizing a combination of cues, including masks, optical flow, rigid flow, and motion region masks, the module identifies genuine moving regions and produces corresponding motion probability maps. The feature points dynamically extracted undergo filtration based on the motion probability map, ensuring that only those feature points outside the motion regions contribute to the computation of the precise camera pose.

The RSO-SLAM system demonstrates remarkable robustness in situations involving local motion of non-rigid objects, the presence of unexpected moving objects, or low-motion states of predefined moving objects. In response to these scenarios, the system promptly adapts the motion probabilities associated with the relevant regions, enhancing its ability to withstand challenges and maintain performance.

To enhance the real-time performance of the system while maintaining localization accuracy, we select the YOLOv5-seg [28] model as the instance segmentation model and the LiteFlowNet2 [29] model as the optical flow prediction model.

IV. MOTION REGION DETECTION

A. Algorithm Procedure

The primary goal of the motion region detection method in RSO-SLAM is to identify regions with significant motion. This is achieved by analyzing dense optical flow data from the optical flow network and the motion mask from the instance segmentation network. The KMC algorithm segments pixels in optical flow images based on motion and spatial relationships, with resulting classes serving as minimal units

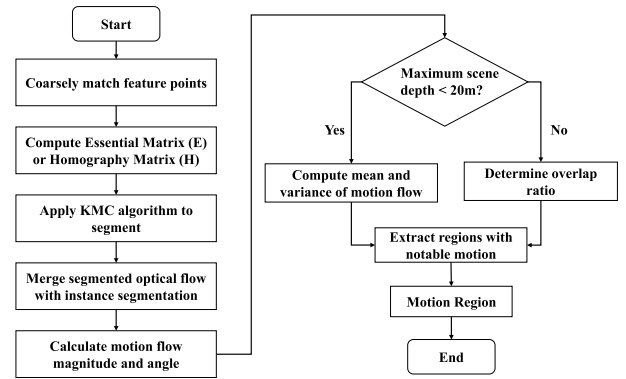


Fig. 2. Motion region detection algorithm flowchart. The unified algorithm for indoor and outdoor scenes takes image sequences as input and outputs the motion regions for each frame.

for delineating significant motion regions. Simultaneously, the instance segmentation algorithm identifies classes related to potentially moving objects like cars and people. Pixels associated with instance segmentation classes are attributed higher initial motion probabilities in the detection and calculation of significant motion regions compared to other pixels. The algorithm flowchart is presented in Fig. 2.

The dense optical flow, generated by the LiteFlowNet2 network, represents a two-dimensional vector field describing pixel displacements in the u and v directions between consecutive frames. Given that a pixel at (x_1, y_1) in the initial frame corresponds to a pixel at (x_2, y_2) in the subsequent frame, the optical flow is mathematically defined as:

$$(u, v) = (x_2 - x_1, y_2 - y_1) \quad (1)$$

Eq. 1 illustrates that optical flow visually portrays the movement of individual pixels between the current and subsequent frames. In scenarios where the camera remains stationary, the identification of moving objects from the optical flow can be accomplished using a straightforward thresholding approach. However, in cases where the camera undergoes complex motion, it is essential to initially compute the rigid flow induced by the camera's internal movement.

B. Coarse Matching

The motion model projects static feature points from the previous frame onto the current frame to identify matching points and minimize reprojection error, thus optimizing the initial pose estimation. In the presence of dynamic feature points in the current frame, a coarse matching strategy is employed, considering both current frame feature points and static feature points from the previous frame. This approach, distinct from full matching, focuses solely on static feature points from the previous frame, reducing the influence of dynamic objects on camera pose estimation. This yields a more accurate transformation matrix, which benefits motion region detection in the current frame.

The effectiveness of coarse matching is illustrated in Fig. 3(a), showcasing how excluding dynamic points from the previous frame prevents erroneous matches with most dynamic feature points in the current frame, particularly those

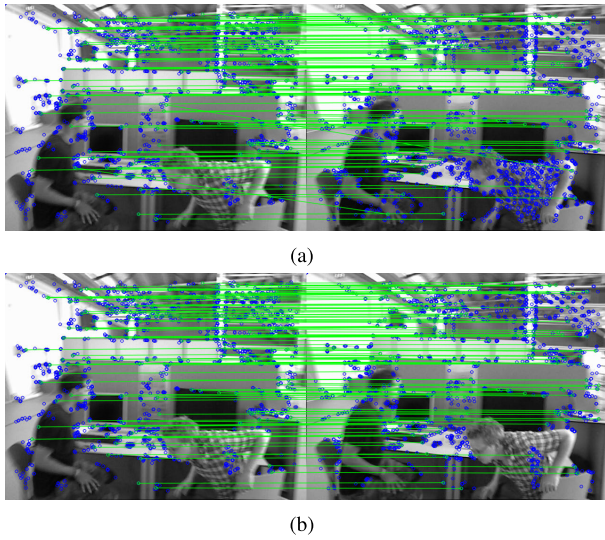


Fig. 3. (a) Coarse matching, (b) Refined matching. Blue points represent detected feature points, while green lines indicate the matching relationships between feature points.

linked to moving entities, thereby ensuring relatively reliable matching relationships. Although there are some mismatches compared to the refined matching achieved after removing dynamic feature points in the current frame (Fig. 3(b)), coarse matching is sufficient for estimating the coarse camera pose.

After the camera transformation matrix $T_{t \rightarrow t+1}$ between consecutive frames is computed, the pixel coordinates of the current frame are projected onto the next frame:

$$p_{t+1}^i = K T_{t \rightarrow t+1} d_t^i K^{-1} p_t^i \quad (2)$$

In the above equation, K represents the camera's intrinsic matrix, p_t^i and p_{t+1}^i are the pixel coordinates in the current frame and the next frame, respectively, and d_t^i corresponds to the depth value of the pixel point. Depth cameras integrated into SLAM systems allow direct retrieval of depth information for pixel points from the depth map. However, for monocular and stereo SLAM systems, incorporating additional depth estimation networks would introduce a real-time performance trade-off. Thus, in monocular and stereo SLAM systems, the homography matrix H is employed for rigid flow computation.

The homography matrix facilitates the mapping between points, enabling the determination of precise pixel point locations on another image without relying on depth information. Although the homography matrix typically assumes coplanar scene points, it can still be utilized when the camera's translation distance relative to the scene depth is small. The homography matrix H is directly computed between the matched pixel points of consecutive frames, and it is then used to project the pixel points of the current frame onto the next frame:

$$\begin{bmatrix} p_{t+1}^i \\ x_2 \\ y_2 \\ 1 \end{bmatrix} = H p_t^i \begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix} \quad (3)$$

Once we obtain the projected positions of each pixel point in the current frame onto the next frame, we can compute the

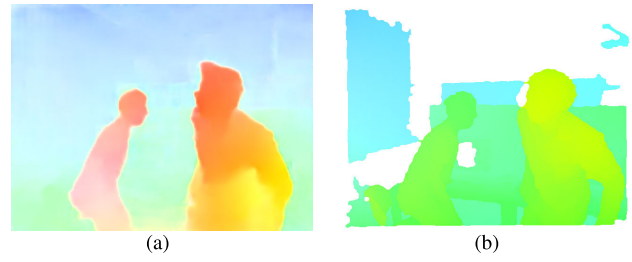


Fig. 4. (a) Optical flow, (b) Rigid flow.

rigid flow f_t^r caused by the camera's motion with Eq. 4. The difference between optical flow and rigid flow can be seen in Fig. 4. Rigid flow reflects the overall motion region of the camera, without paying attention to the motion of each region.

$$f_t^r = p_{t+1}^i - p_t^i, \quad i \in I \quad (4)$$

C. Optical Flow Classification

Calculation errors in optical flow, whether from dense optical flow or rigid flow, often arise due to factors such as scene texture loss and camera motion blur. Using raw optical flow data directly for motion region extraction can introduce significant noise, compromising accuracy. To address this, we employ a clustering approach to group optical flow regions, reducing error impact.

Directly using pixel-wise displacements (u and v directions) for optical flow classification has exhibited discrepancies compared to subjective judgments. Consequently, we adopt a two-step approach: first, we visualize optical flow and generate RGB representations as the basis for classification. However, the motion state of scene objects varies over time, resulting in diverse optical flow patterns. Inadequate categorization challenges the accurate representation of distinct scene components' motion states, while excessive categorization hinders noise reduction.

In this paper, we propose an adaptive segmentation method called KMC algorithm to classify optical flow. Due to limited color information in visualized optical flow images, we utilize the k-means method [37] for RGB image classification based on color. Initially, we group pixels with similar motion direction and magnitude, aligning with subjective judgments. It's important to note that some grouped regions may represent distinct entities, necessitating individual processing.

Algorithm 1 outlines the procedure, with the initial classification into K classes being adaptively determined based on color dispersion. In the subsequent step, we establish binary matrices for each class and evaluate the results. Connectivity assessment separates nonadjacent regions into new categories. This qualitative classification, based on region-specific motion characteristics, yields superior outcomes for motion region detection in subsequent analyses.

Based on Fig. 5, it is evident that the optical flow classification algorithm proposed in this study successfully categorizes optical flow by considering both similarity and spatial connectivity. As a result, the obtained classification results are following human subjective judgment.

Algorithm 1 KMC Algorithm

Randomly initialize K cluster centers: μ .
Assign labels to samples based on nearest cluster centers:
 $c_i = \arg \min_k \|x_i - \mu_k\|^2$, x_i is the vector corresponding to each pixel point
Update cluster centers as the mean of samples in each cluster:
 $\mu_k = \frac{1}{|Set_k|} \sum_{i \in Set_k} x_i$
repeat
 Assign labels to samples based on updated cluster centers.
 Update cluster centers as the mean of samples in each cluster.
until cluster centers converge
Obtain an initial labeled image L_{ini} with K labels.
for i in $Range(K)$ **do**
 Obtain binary image I_i : $I_i(x, y) = (L(x, y) == i)$
 Initialize an output labeled image L_i .
 for each pixel $I_i(x, y)$ in I_i **do**
 if $I_i(x, y) = 0$ **then**
 Set $L_i(x, y) = 0$ (background).
 else
 Retrieve neighboring pixel labels: $L_i(x-1, y)$,
 $L_i(x, y-1)$, $L_i(x-1, y-1)$, $L_i(x+1, y-1)$.
 if all neighboring pixel labels are 0 **then**
 Assign a new unique label to the current pixel.
 else
 Assign the current pixel the smallest label
 among its neighbors.
 Combine labels into the final labeled image L_{final} :

$$L_{final}(x, y) = L_{ini}(x, y) + L_{L_{ini}(x, y)}(x, y)$$



Fig. 5. Result of optical flow classification: (a) Before classification, (b) After classification.

Before performing the actual optical flow classification, it is necessary to assess the scene depth. If the maximum depth of scene feature points exceeds 20m, the scene is considered to be an outdoor scene. In indoor scenes, the classification of optical flow directly relies on the visualization image of optical flow. However, in outdoor scenes, the motion of distant objects appears less prominent due to smaller flow magnitudes. Therefore, in outdoor scenes, we choose to use the angle map of the optical flow as the basis for optical flow classification.

As shown in Fig. 6, when examining the visualization image of optical flow, the motion of a car is not visible. However, in the angle map, the motion direction of the car exhibits a distinct difference from the background, making the motion of the car particularly evident. By applying the KMC segmentation method to the angle map, it becomes possible to effectively separate the regions containing moving cars from the background.

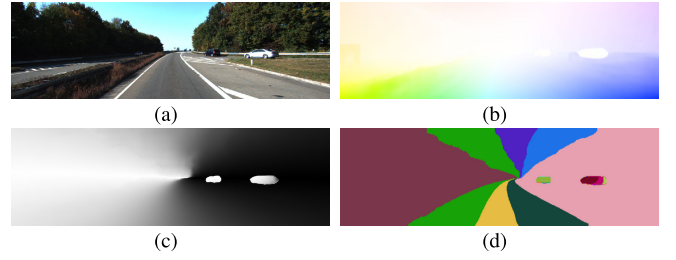


Fig. 6. Result of angle-based optical flow classification: (a) Original image, (b) Optical flow, (c) Angle map, (d) Classified angle map.



Fig. 7. (a) Angle map, (b) Magnitude map. The angle map conveys the disparity in motion direction between the dynamic object and the background, whereas the magnitude map delineates the amplitude of motion exhibited by the dynamic object.

D. Significant Movement Region

Once the optical flow and the rigid flow are obtained, it is possible to preliminarily determine which regions in the current scene belong to the moving areas.

First, we need to obtain the angle map I_{ang} and the magnitude map I_{nm} of the motion flow based on the optical flow f and the rigid flow f^r :

$$f_i = (u_i, v_i), \quad i \in f \quad (5)$$

$$f_i^r = (u_i^r, v_i^r), \quad i \in f^r \quad (6)$$

Calculate the angle map:

$$I_{ang}[i] = \arccos \left(\frac{f_i \cdot f_i^r}{\|f_i\| \times \|f_i^r\|} \right), \quad i \in I_{ang} \quad (7)$$

Obtain the difference vector for each flow and subsequently obtain the magnitude map:

$$\Delta f_i = f_i - f_i^r = (\Delta u_i, \Delta v_i), \quad i \in I_{nm} \quad (8)$$

$$I_{nm}[i] = \begin{cases} 0, & \text{Norm}(\Delta f_i) > Nm_{ua}^{th} \\ Nm_{max}^{th}, & Nm_{max}^{th} < \text{Norm}(\Delta f_i) \leq Nm_{ua}^{th} \\ \text{Norm}(\Delta f_i), & 0 \leq \text{Norm}(\Delta f_i) < Nm_{max}^{th} \end{cases} \quad (9)$$

Here, $Norm()$ denotes the magnitude of a vector, Nm_{ua}^{th} represents the threshold for determining the unreliability of the magnitude value at a certain point, and Nm_{max}^{th} represents the maximum magnitude value set for a point.

As shown in Fig. 7, the angle map and magnitude map provide a more intuitive indication of the objects in the scene that exhibit prominent motion. Additionally, the angle map and the magnitude map are normalized to the range of 0-255 for visualization and subsequent thresholding.

In the previous phase, we obtained a set of distinct regions, labeled as S , through the classification of optical flow. Concurrently, we acquired the magnitude map and angle map

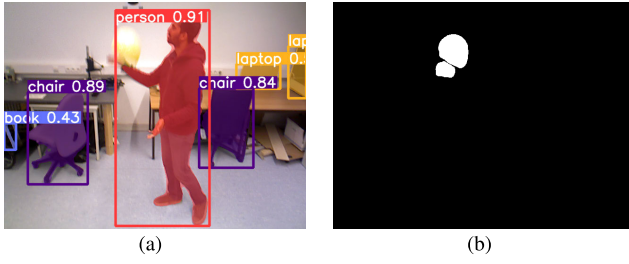


Fig. 8. (a) Result of instance segmentation, (b) Significant movement region. The significant movement region in the right image corresponds to the region of the balloon and hand in the left image.

of motion flow. Following this, it is crucial to compute the mean and variance of both the magnitude map and angle map for each corresponding region. These computed measures will then be used as criteria for identifying and isolating the motion regions.

In the context of each class S_i within the ensemble S , we undertake the computation of pixel counts denoted as S^{Num} , followed by the derivation of the mean maps, S_{nm}^{Mn} and S_{ang}^{Mn} , corresponding to the magnitude map and angle map. Furthermore, we engage in the calculation of the variance set, S_{nm}^{Var} , particularly for the magnitude map:

$$S_{nm}^{Mn}[i] = \frac{\sum_{(x,y) \in S_i} (I_{nm}(x, y)/20)}{S^{Num}[i]}, \quad i \in S \quad (10)$$

$$S_{ang}^{Mn}[i] = \frac{\sum_{(x,y) \in S_i} I_{ang}(x, y)}{S^{Num}[i]}, \quad i \in S \quad (11)$$

$$S_{nm}^{Var}[i] = \frac{\sum_{(x,y) \in S_i} (\frac{I_{nm}(x,y)}{20} - S_{nm}^{Mn}[i])^2}{(S^{Num}[i] - 1)}, \quad i \in S \quad (12)$$

From the set S , we selectively identify regions that adhere to the criteria reflecting motion characteristics defined by S_{nm}^{Mn} , S_{ang}^{Mn} , and S_{nm}^{Var} . The key aim of this step is to extract objects in motion that might not be captured by the instance segmentation model. Thus, our specific focus lies in the extraction of regions demonstrating significant motion, denoted as M_{mv} .

By extracting the pixels that meet the conditions for each class in the set, we can obtain the significant moving region M_{mv} . This allows us to identify the areas in the scene where motion is more evident and also addresses the issue of instance segmentation methods being unable to detect undefined moving objects. As shown in Fig. 8, we can see that the instance segmentation model did not detect the balloon. However, through our method, we were able to detect both the balloon and the person's hand, which have significant motion.

The instance segmentation network provides a mask denoted as M_{obj} , which represents potential moving objects. Additionally, the mask M_{mv} is obtained through motion region detection, representing the regions with motion. By combining these two masks, we obtain the set of all potential moving regions denoted as M_{mv}^{all} in the current frame.

In scenarios where the maximum depth value in the scene exceeds 20m, directly utilizing the magnitude map to detect significant moving regions proves ineffective. To address this issue, this paper introduces a more direct yet effective approach. In outdoor scenes, potential moving regions mainly consist of two categories: predefined potential moving objects detected by instance segmentation and regions classified based

on the direction map with an area smaller than 0.05 times the total image area. These categories collectively form the sets of potential moving objects denoted as S^{out} . Among these sets, for any two regions A and B , we calculate the overlap ratio $OVER_{AB}$ using the following formula:

$$OVER_{AB} = \frac{A \cap B}{\min(A, B)} \quad (13)$$

If $OVER_{AB}$ is greater than 0.7, we consider the region with the smaller area between A and B as the significant moving region, denoted as M_{mv} . The value 0.7 is determined empirically through experimentation, but it holds generally true for most scenarios. The overlap rate calculation highlights the need to utilize instance segmentation for obtaining individual object masks, a requirement not fulfilled by semantic segmentation. The sum of all the regions in the set S^{out} represents the mask of potential moving objects, denoted as M_{mv}^{all} .

V. MOTION PROBABILITY CALCULATION

A. Algorithm Procedure

The motion probability calculation in RSO-SLAM aims to compute the motion probabilities for individual regions within the scene based on the detected potential motion regions. This computation facilitates the identification of regions that contain dynamic feature points. Algorithm 2 presents the comprehensive workflow for motion probability calculation.

Algorithm 2 Motion Probability Calculation

- 1 Initialize M_{mv}^{ini} by combining notable and potential motion: $M_{mv}^{ini} = \lambda_{mv} M_{mv} + \lambda_{all} M_{mv}^{all}$
 - 2 Apply optical flow attenuation propagation to compute M_{trans} from the previous frame:
 $M_{trans}^{t+1}(x + f^t(x, y)[0], y + f^t(x, y)[1]) = M_p^t(x, y)$
 - 3 Introduce motion continuity constraints using M_{trans} to compensate for missed motion detections.
 - 4 **if** Maximum scene depth $\leq 20m$ **then**
 - 5 Generate Motion Probability Map:
 $M_p = M_{mv}^{ini} + M_{trans} + M_{nm}$
 - 6 Filter and remove dynamic feature points based on M_p , M_{nm} , and M_{ang} .
 - 7 **else**
 - 8 Calculate epipolar error: $E = \|p - \hat{p}\|$
 - 9 Calculate mean motion flow magnitude Nm .
 - 10 Calculate depth Dp for each region in M_{mv}^{all} :
 $Dp = \frac{1}{N_r} \sum_{i=1}^{N_r} Depth(i)$,
 - 11 Compute the candidate motion region map M_{mv}^{can} based on E , Nm , and Dp .
 - 12 Generate Motion Probability Map:
 $M_p = M_{mv}^{ini} + M_{trans} + M_{mv}^{can}$.
 - 13 Filter feature points using M_p to remove dynamic features.
-

In practical applications, the optical flow we can obtain represents the pixel motion from the previous frame to the current frame. Therefore, the extracted M_{mv} represents the

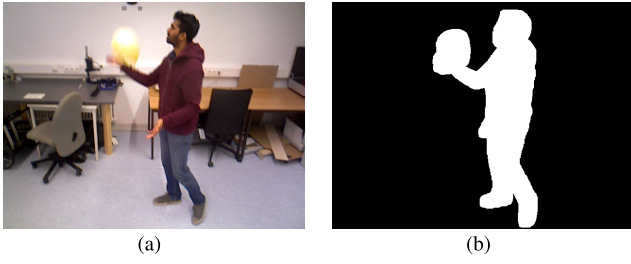


Fig. 9. (a) Original image, (b) Initial Motion mask.

motion region from the previous frame. We need to propagate the motion regions from the previous frame to the current frame based on the optical flow, which can be expressed as:

$$\begin{aligned} M_{mv}^t(x + \text{flow}(x, y)[0], y + \text{flow}(x, y)[1]) \\ = M_{mv}^{t-1}(x, y), \quad (x, y) \in \text{flow} \end{aligned} \quad (14)$$

To facilitate subsequent motion probability calculations and inter-frame propagation, we combine the two binary images M_{mv} and M_{mv}^{ini} into a matrix M_{mv}^{ini} in the range of 0-255 using Eq. 15. The values of λ_{mv} and λ_{all} are established to align with the motion probability calculation approach.

$$M_{mv}^{ini} = M_{mv} \times \lambda_{mv} + M_{mv}^{all} \times \lambda_{all} \quad (15)$$

After obtaining the initial motion mask, to prevent feature points on the boundaries of moving objects from being considered static, we further perform morphological dilation on the motion mask. The final mask is shown in Fig. 9.

By employing this approach, the generated mask effectively encompasses the regions in the scene where motion can occur. The mask assigns a probability value to each pixel, indicating the likelihood of that pixel being in a motion state to some degree. While using this mask directly as the motion region can successfully eliminate all dynamic feature points, it may also result in the removal of certain static feature points. Therefore, it becomes necessary to compute the motion probability for each feature point to refine the classification.

B. Optical Flow Propagation

Taking into account the persistence of object motion and the potential missed detections in both the instance segmentation network and the motion region detection based on optical flow, this paper introduces a method for motion probability attenuation propagation using optical flow.

In the initial frame of the system, the motion regions are obtained solely through instance segmentation. From the second frame onwards, the system initiates the propagation of motion probability. This involves subtracting a predefined value from the motion region mask obtained in the current frame and propagating it to the subsequent frame according to the optical flow model. However, this step needs to be executed after calculating the optical flow for the next frame. Therefore, we perform attenuation in the current frame and then propagation in the next frame according to the following equations:

$$\begin{aligned} M_p^t(x, y) &= M_p^t(x, y) - N_{decrease} \\ N_{decrease} &= 255 \times \frac{8}{N_{fr}} \end{aligned} \quad (16)$$

where N_{fr} signifies the video frame rate. $N_{decrease}$ is designed to maintain a consistent propagation period for motion probability across different frame rates.

In the next frame, the propagated mask M_{trans}^{t+1} is obtained using the following equation:

$$\begin{aligned} M_{trans}^{t+1}(x + \text{flow}^t(x, y)[0], y + \text{flow}^t(x, y)[1]) \\ = M_p^t(x, y), \quad (x, y) \in \text{flow} \end{aligned} \quad (17)$$

Consequently, starting from the second frame, the system possesses the initial motion mask M_{mv-ini} and the propagated mask M_{trans} .

The optical flow attenuation propagation strategy not only helps calculate the motion probability but also compensates for missed detections of moving objects. In dynamic scenarios, challenges such as partial visibility of objects in the camera's field of view, object motion, and image blur due to camera rotation significantly impede object segmentation, resulting in missed detections and consecutive frame omissions. This strategy represents a straightforward yet potent methodology. It involves the propagation of the motion probability map with decay, relying on the continuity of motion, and imposes minimal impact on real-time performance.

Fig. 10 illustrates the results of missed detection compensation in the aforementioned situations, including incomplete appearance (a), object motion (b), image blur (c), and consecutive frame omissions (d). The first row of images exhibits the instance detection results achieved through YOLOv5-seg, where the identified regions are highlighted in green. Subsequently, the second row demonstrates the outcomes of the proposed method for detecting potential motion regions after compensating for missed detections, with the regions highlighted in blue.

C. Indoor Scene

In the case of scenes with a maximum depth of less than 20m, such as indoor scenes, the motion flow amplitude map can be utilized to refine the motion probability at different pixel locations. However, since the previously obtained I_{nm} corresponds to the previous frame, it needs to be propagated to the current frame using optical flow to derive the motion probability offset matrix, denoted as M_{nm} .

By combining the obtained masks, we obtain M_p as follows:

$$M_p = M_{mv}^{ini} + M_{trans} + M_{nm} \quad (18)$$

The pixel values in M_p range from 0 to 255. If the value corresponding to a feature point in M_p exceeds $N_{decrease}$, it is considered to be in a motion state. This implies that regions detected solely by instance segmentation will not be directly classified as motion regions. Instead, they require the region to be identified as an evident motion region, exhibit a significant motion amplitude, or be in a motion state in the previous frame. This approach enables the direct and effective computation of regions that genuinely exhibit motion. Fig. 11 illustrates the potential motion regions in blue, where regions with higher motion probabilities are highlighted in red. The proposed method can accurately detect regions experiencing

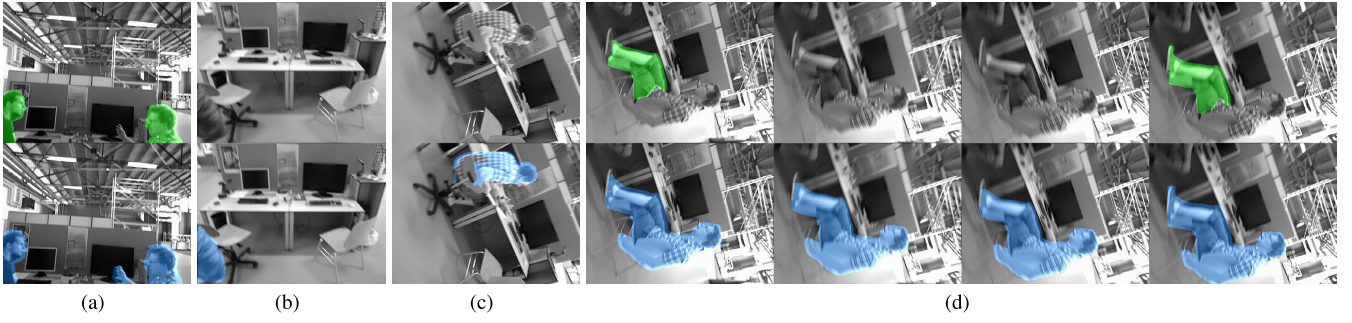


Fig. 10. Illustrations of missed object detection instances and the corresponding compensation results. In the images, the green region denotes individuals detected by the YOLOv5 segmentation method, while the blue region indicates potential motion areas identified by RSO-SLAM. The scenarios include (a) Partial visibility of objects within the camera's field of view, (b) Blurred image conditions, (c) Unusual camera angle leading to singular perspectives, (d) Continuous frame-level missed object detection occurrences.

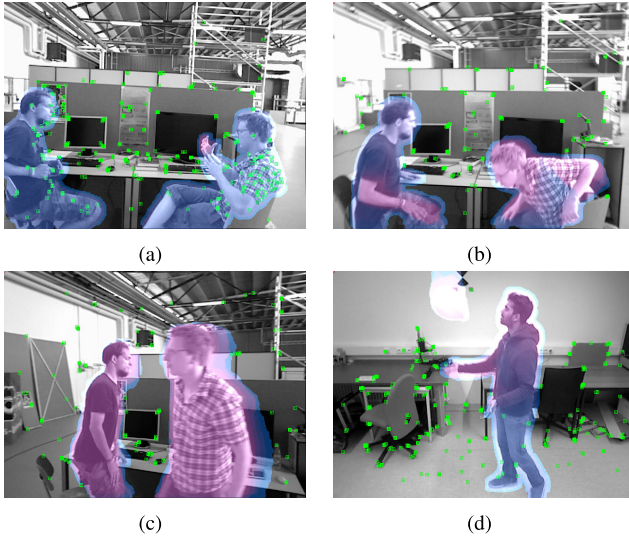


Fig. 11. Results of motion probability calculation in varied dynamic environments: (a) Low Dynamic, (b) Partial Motion, (c) High Dynamic, and (d) Complex Motion. Blue represents potential motion areas, while red indicates actual motion areas with shading intensity reflecting motion probability magnitude.

motion, regardless of whether it is a high dynamic or low dynamic scene.

Certainly, for feature points in which the corresponding value in M_p is less than $N_{decrease}$, an additional evaluation is necessary considering the motion angle map M_{ang} and magnitude map M_{nm} associated with the feature point in the previous frame.

The probability of the point being classified as a motion point is defined as follows:

$$P_{mv}(x_t, y_t) = \begin{cases} 1 & M_p(x_t, y_t) > N_{decrease} \\ 1, & \frac{M_{ang}(x_{t-1}, y_{t-1})}{20} + \frac{M_{nm}(x, y)}{20} > 3 \\ 0, & \text{other} \end{cases} \quad (19)$$

D. Outdoor Scene

For outdoor scenes, the reliability of using M_{nm} to determine the motion status diminishes. Therefore, slight adjustments are required in the method for detecting motion regions. To determine the motion status of regions, we analyze them using motion consistency and motion magnitude.

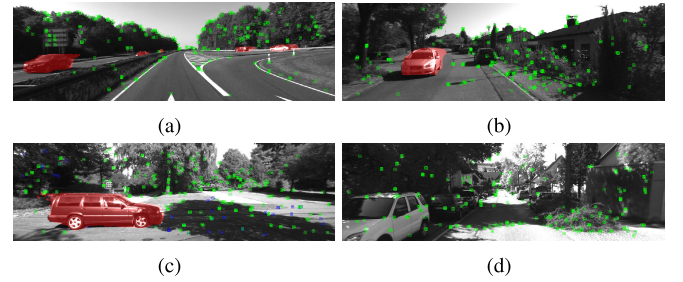


Fig. 12. Motion region detection in outdoor scenes: (a) High Dynamic Scenes, (b) Partial Motion Scenes, (c) Cross Camera Scenes, (d) Static Scenes. Motion Regions are marked in red.

Before performing the analysis, it is necessary to estimate the depth of these object regions since different depths result in significant variations in optical flow magnitudes. The depth of an object is determined by calculating the mean depth of all feature points located on the object. If no feature points are found within an object, it is considered a static object. Motion consistency is assessed by evaluating epipolar geometry constraints. We obtain the fundamental matrix using EPnP and compute the epipolar lines for the current frame using the fundamental matrix. The average distance from all matched points within the object to their corresponding epipolar lines is computed as E , which serves as the criterion for motion consistency. The motion magnitude N_m of the object is obtained using the same method as before.

For regions that are determined to be in motion, the corresponding values in the mask are set to $N_{decrease} + 20$, resulting in M_{mv}^{can} . By combining M_{mv}^{ini} , M_{trans} , and M_{mv}^{can} , we obtain the motion probability map M_p .

$$M_p = M_{mv}^{ini} + M_{trans} + M_{mv}^{can} \quad (20)$$

When determining the motion status of feature points, it is sufficient to check if the value corresponding to the feature point's position in M_p is greater than $N_{decrease}$. If so, the feature point is considered to be in motion state.

The proposed method effectively detects the regions that are truly in a motion state in various dynamic scenes in outdoor environments, as can be observed from the result images shown in Fig. 12. In the first image, all cars are identified as being in motion. In the second image, only the car on the left is detected as being in motion. In the third image, the car

TABLE I
ATE COMPARISON OF TUM IN RGB-D MODE[M]

Sequence	ORB-SLAM2	Dyna-SLAM	DS-SLAM	Blitz-SLAM	DGS-SLAM	CFP-SLAM	RSO-SLAM
	RMSE	RMSE	RMSE	RMSE	RMSE	RMSE	RMSE
fr3/s/xyz	0.0092	0.0127	/	0.0148	0.0092	<u>0.0090</u>	0.0088
fr3/s/half	0.0192	0.0186	/	0.0160	0.0182	0.0147	0.0149
fr3/s/static	0.0087	/	0.0065	/	0.0057	0.0053	0.0062
fr3/s/rpy	<u>0.0195</u>	/	/	/	/	0.0253	0.0189
fr3/w/xyz	0.7214	0.0164	0.0247	0.0153	0.0156	0.0141	0.0146
fr3/w/half	0.4667	0.0296	0.0303	0.0256	0.0301	0.0237	0.0234
fr3/w/static	0.3872	0.0068	0.0081	0.0102	<u>0.0059</u>	0.0066	0.0059
fr3/w/rpy	0.7842	0.0354	0.4442	0.0356	0.0301	0.0368	<u>0.0330</u>
Top-2 Count (Top1)	1(0)	0(0)	0(0)	0(0)	2(1)	5(3)	8(4)

is observed to pass through the field of view. In the last image, all cars are determined to be stationary.

The proposed approach has successfully determined the motion status of all feature points in both indoor and outdoor scenes. This comprehensive methodology effectively addresses the challenge of missed detections in instance segmentation and optical flow analysis results, while also reducing the likelihood of incorrectly categorizing stationary feature points as dynamic. As a result, the system's performance is significantly improved, leading to enhanced robustness.

VI. EXPERIMENT RESULTS

A. Experimental Setup

The experimental platform employed in this study comprises an Intel Core i7-12700H 3.5GHz CPU and an RTX3070 graphics card boasting 8GB of VRAM, operating on Ubuntu 20.04. The dynamic SLAM system is implemented using C++, with separate threads dedicated to optical flow detection and instance segmentation developed in Python. During the real-world experiment, we captured data at a frame rate of 30 FPS using a handheld experimental platform equipped with a ZED2i Stereo Camera. In this paper, the experiments are conducted using the TUM dataset [30], BONN dataset [31], and KITTI dataset [32]. These three datasets are widely used for testing dynamic SLAM systems. The TUM and BONN datasets both are indoor datasets, but the BONN dataset contains moving objects that are not detectable by common instance segmentation models. The KITTI dataset is used for outdoor scenes in stereo mode. These three datasets include the primary scenarios in which dynamic SLAM methods are commonly applied. To ensure real-time processing capabilities, the image frames are downsampled during both motion region detection and motion probability calculation stages.

The TUM dataset is characterized by two distinct motion categories: high dynamics (referred to as “w” for walking) and low dynamics (referred to as “s” for sitting). In the high-dynamics scenario, individuals traverse around a table, resulting in larger motion regions. Conversely, in the low dynamics scenario, people are seated on chairs and engaged in conversation, yielding smaller motion regions. Each scenario encompasses four distinct camera motion trajectories, namely xyz, halfsphere, rpy, and static. In contrast, the BONN dataset captures indoor scenes with a greater diversity of motion sequences, including activities such as popping balloons or

moving boxes, thereby introducing more complex motion scenarios compared to the TUM dataset. On the other hand, the KITTI dataset is collected using a multisensor vehicle platform equipped with a camera, high-precision GPS/IMU navigation system, and lidar. This dataset captures real-world outdoor traffic scenarios, encompassing highways, urban roads, and rural roads with varying road conditions.

To test the effectiveness and generality of the proposed RSO-SLAM, experiments are conducted in both RGBD and monocular modes on the BONN and TUM datasets, and in stereo mode on the KITTI dataset. These tests cover nearly all the scenarios explored in dynamic SLAM research. The quantitative comparison results are presented in Tables I, II, III, IV, V and VI. The absolute trajectory error (ATE) is used to evaluate the localization accuracy of these algorithms. ATE is used to assess the global consistency of the estimated trajectory and indicates the difference between the camera pose estimates and the ground truth for each frame. The Relative Pose Error (RPE) serves as a metric for assessing the precision of estimated trajectories by quantifying local rotational or translational errors. It measures the dissimilarity between the estimated pose transformation matrix, derived between successive frames, and the actual pose change matrix. The root mean square error (RMSE) is used as the evaluation metric for each error table. The “Top-2 Count (Top1)” denotes the count of optimal items achieved by the method, representing instances in the dataset where the method performs optimally or is the top choice. The value in the parentheses specifically denotes the count of instances in which the method attains the highest level of performance.

B. Comparison With State-of-the-Arts

Tables I and II present the comparison of RSO-SLAM with the classical SLAM algorithm ORBSLAM2 [3], the classical dynamic SLAM algorithms (Dyna-SLAM [27], DS-SLAM [26]), and the state-of-the-art dynamic SLAM algorithms (Blitz-SLAM [35], DGS-SLAM [17], CFP-SLAM [16]) on the TUM dataset in RGBD mode. The superior RMSE metrics are highlighted in bold, while the second-best RMSE metrics are underlined in the table. The subsequent tables follow the same annotation style. The ATE and RPE plots of RSO-SLAM on the eight sequences are shown in Fig. 13.

Based on the experimental findings using the TUM dataset, RSO-SLAM demonstrates remarkable robustness in environments with both low dynamics and high dynamics.

TABLE II
RPE COMPARISON OF TUM IN RGB-D MODE[M]

Sequence	ORB-SLAM2	Dyna-SLAM	DS-SLAM	Blitz-SLAM	DGS-SLAM	CFP-SLAM	RSO-SLAM
	RMSE	RMSE	RMSE	RMSE	RMSE	RMSE	RMSE
fr3/s/xyz	0.0117	0.0142	/	0.0144	0.0134	0.0114	0.0118
fr3/s/half	0.0231	0.0239	/	0.0165	0.0276	0.0162	0.0191
fr3/s/static	0.0090	/	0.0078	/	0.0082	0.0072	0.0079
fr3/s/rpy	0.0245	/	/	/	/	0.0316	0.0238
fr3/w/xyz	0.3944	0.0217	0.0333	0.0197	0.0228	0.0190	0.0185
fr3/w/half	0.3480	0.0284	0.0297	0.0253	0.0366	0.0259	0.0269
fr3/s/static	0.2349	0.0089	0.0102	0.0129	0.0101	0.0089	0.0075
fr3/w/rpy	0.4582	0.0448	0.1503	0.0473	0.0432	0.0500	0.0421
Top-2 Count (Top1)	2(0)	1(0)	1(0)	1(1)	1(0)	5(3)	4(4)

TABLE III
ATE COMPARISON OF BONN IN RGB-D MODE[M]

	Sequence	ORB-SLAM2	StaticFusion	ReFusion	DynaSLAM	DGS-SLAM	RSO-SLAM
Low Dynamic	balloon_tracking	0.0331	0.221	0.302	0.0418	0.0321	0.0301
	balloon_tracking2	0.0290	0.366	0.322	0.0311	0.0277	0.0288
	kidnapping_box	0.0267	0.336	0.148	0.0296	0.0252	0.0251
	kidnapping_box2	0.0232	0.263	0.161	0.0240	0.0215	0.0239
	moving_no_box2	0.0319	0.364	0.179	0.0297	0.0281	0.0326
	placing_no_box2	0.0276	0.177	0.141	0.0199	0.0204	0.0184
	removing_no_box	0.0163	0.136	0.041	0.0166	0.0149	0.0153
	removing_no_box2	0.0202	0.129	0.111	0.0208	0.0199	0.0206
Top-2 Count (Top1)		1(0)	0(0)	0(0)	2(0)	7(5)	6(3)
High Dynamic	balloon	0.069	0.233	0.175	0.0302	0.0228	0.0248
	balloon2	0.112	0.293	0.254	0.0248	0.0244	0.0244
	crowd	1.41	3.586	0.204	0.0163	0.0176	0.0166
	crowd2	0.6027	0.215	0.155	0.0261	0.0226	0.0229
	crowd3	0.5243	0.168	0.137	0.0230	0.0241	0.0310
	moving_no_box	0.4181	0.141	0.071	0.0282	0.0180	0.0201
	moving_o_box	0.3901	0.331	0.343	LOST	0.2463	0.2655
	moving_o_box2	0.6216	0.309	0.528	LOST	0.3065	0.2690
	person_tracking	0.5791	0.484	0.289	0.0383	0.0609	0.0366
	person_tracking2	0.7681	0.626	0.463	0.0295	0.0484	0.0370
	placing_no_box	0.9027	0.125	0.106	0.1401	0.0158	0.0155
	placing_no_box3	0.1337	0.256	0.174	0.0670	0.0337	0.0250
	placing_o_box	0.2965	0.330	0.571	LOST	0.2403	0.2482
	removing_o_box	0.3741	0.334	0.222	0.2466	0.2684	0.2876
	synchronous	0.8284	0.446	0.41	0.0955	0.0393	0.0097
	synchronous2	1.2717	0.027	0.022	0.0065	0.0063	0.0061
Top-2 Count (Top1)		0(0)	1(0)	1(1)	4(2)	12(5)	14(8)

It surpasses both classical dynamic SLAM algorithms and the most recent SLAM algorithms in terms of performance. Notably, RSO-SLAM achieves superior results in the “s/half,” “s/rpy,” “w/half,” and “w/static” sequences of the TUM dataset. It outperforms the current state-of-the-art methods, namely CFP-SLAM and DGS-SLAM, and achieves results that are comparable to the best metrics in other sequences. Compared to the impressive performance of CFP-SLAM on the TUM dataset, RSO-SLAM demonstrates overall advantages.

The BONN dataset offers a more diverse range of moving objects in comparison to the TUM dataset, and certain methods that perform well on the TUM dataset may not be as effective on the BONN dataset. For example, the CFP-SLAM method, which demonstrates outstanding performance in the TUM data set, encounters challenges in the scenarios presented in the BONN data set. Therefore, in addition to comparing RSO-SLAM with the classical ORB-SLAM2 method, we also compare it with the latest methods (StaticFusion [8],

ReFusion [38], DynaSLAM [27], DGS-SLAM [17]) that provide actual test results on the BONN dataset, as reported in their original papers. The results of the comparison are depicted in Table III. Based on the experimental findings, it is evident that the RSO-SLAM demonstrates sufficient robustness in complex motion scenarios, such as those encountered in the BONN dataset. Compared to the currently best-performing DGS-SLAM method on the BONN dataset, our proposed method achieves the best or second-best results in multiple experimental sequences, exhibiting high localization accuracy in both low-dynamic and high-dynamic scenes. The test results on the BONN dataset further validate the enhanced robustness and higher accuracy of the RSO-SLAM method in complex motion scenarios.

In the monocular mode, the majority of methods evaluated in the RGBD mode are not applicable. We perform a comparative analysis among RSO-SLAM, ORB-SLAM2, DynaSLAM, and the state-of-the-art SOF-SLAM [24] methods. The results of the comparison in the monocular mode in the TUM data set

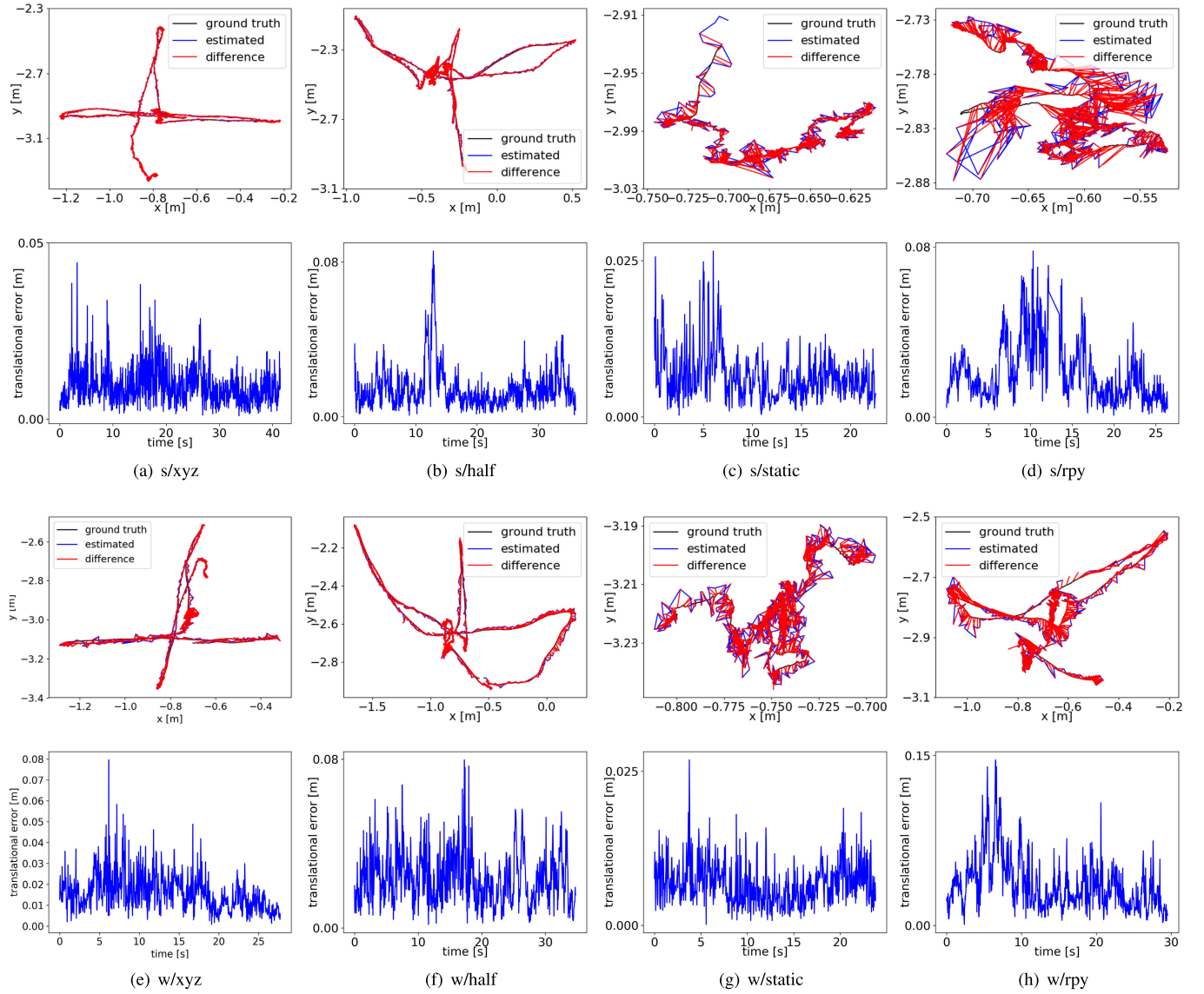


Fig. 13. ATE and RPE of TUM in RGB-D mode from RSO-SLAM.

TABLE IV
ATE COMPARISON OF TUM IN MONOCULAR MODE[M]

Sequence	ORB-SLAM2		DynaSLAM		SOF-SLAM		RSO-SLAM	
	RMSE	% traj	RMSE	% traj	RMSE	% traj	RMSE	% traj
fr3/s/xyz	0.0512	76.92	0.0625	95.00	0.0472	95.47	0.0446	95.65
fr3/s/half	0.0230	98.82	0.0322	90.27	0.0340	91.25	0.0240	98.97
fr3/s/static	NO INI	NO INI	NO INI	NO INI	/	19.71	0.0134	53.97
fr3/s/rpy	0.0395	65.12	/	1.70	0.0275	82.66	0.0303	92.20
fr3/w/xyz	0.179	56.92	0.0335	99.06	0.0314	98.66	0.0205	97.34
fr3/w/half	0.0323	86.41	0.0264	87.81	0.0287	99.51	0.0273	99.61
fr3/w/static	0.0108	87.93	/	2.83	0.0125	91.63	0.0125	86.19
fr3/w/rpy	0.0573	81.86	0.0412	88.90	0.0336	80.37	0.0282	80.37

TABLE V
ATE COMPARISON OF BONN IN MONOCULAR MODE[M]

Sequence	ORB-SLAM2		DynaSLAM		SOF-SLAM		RSO-SLAM	
	RMSE	% traj	RMSE	% traj	RMSE	% traj	RMSE	% traj
balloon	/	27.62	0.0516	65.83	0.0645	90.63	0.0564	90.41
balloon_tracking	0.0302	95.58	0.0242	90.49	0.0330	97.96	0.0291	98.13
moving_no_box	0.0214	92.27	0.0256	88.41	0.0205	98.20	0.0144	97.94
person_tracking	0.1010	67.53	/	25.73	/	29.87	0.0772	96.38
placing_no_box	/	15.55	0.0550	92.22	0.0255	97.51	0.0179	93.33
crowd	0.0224	98.38	0.0221	95.25	0.0197	97.31	0.0176	97.73
removing_no_box	NO INI	NO INI	0.0132	97.16	0.0129	90.05	0.0142	93.93
kidnapping_box	0.0811	97.24	0.0223	95.32	0.0227	98.07	0.0206	98.63

are presented in Table IV. Furthermore, the comparison results in monocular mode on the BONN data set are presented in Table V, where we select eight representative sequences from the BONN data set that can operate effectively in monocular mode. The column “% traj” in both tables represents the percentage of the trajectory length obtained in the monocular mode relative to the total trajectory length. This metric is influenced by the speed of initialization and the system’s

stability. The presence of “/” in the table signifies that the trajectory result is too short to possess practical significance for calculating the Absolute Trajectory Error (ATE). When comparing the localization accuracy between different methods, the ATE comparison of the trajectory results is only meaningful when the value of “% traj” exceeds 80%. The term “NOT INI” indicates that the method was unable to achieve initialization for that specific sequence.

Based on the experimental results, it is evident that RSO-SLAM demonstrates effective operation in the monocular mode. In the majority of sequences, the trajectory completeness exceeds 90%, indicating that the method can facilitate SLAM system initialization in dynamic scenes by accurately eliminating motion regions. In terms of positional accuracy, RSO-SLAM consistently outperforms classical methods such as ORB-SLAM2, DynaSLAM, and the state-of-the-art SOF-SLAM across the majority of sequences, underscoring its effectiveness in monocular mode.

To assess the performance of RSO-SLAM in outdoor scenes and stereo mode, comprehensive tests were conducted on the KITTI dataset in stereo mode. The results are presented in Table VI. The experimental findings indicate that our method maintains commendable performance in outdoor dynamic scenes. Given that moving objects typically occupy a smaller region in outdoor scenarios, the improvement in localization accuracy compared to ORB-SLAM2 may not be substantial. Nevertheless, our method still demonstrates noticeable enhancements in most sequences. In comparison to the classical dynamic SLAM method, DynaSLAM, optical flow-based SLAM method, Stereo-DSO [39], as well as the recent dynamic SLAM SOTA methods VDO-SLAM [20] and AirDos in stereo mode, RSO-SLAM exhibits superior accuracy in moving object removal. Our method consistently demonstrates lower errors across the majority of sequences in the KITTI dataset. Notably, RSO-SLAM excels in terms of Relative Pose Error (RPE), showcasing its superior ability to accurately recognize dynamic objects. This ensures that the system remains unaffected by dynamic objects while maintaining an adequate number of feature points for accurate camera pose estimation, particularly in scenarios involving vehicle turning.

C. Ablation Experiments

To evaluate the effectiveness of each module in our algorithm we design a series of ablation experiments, and the experimental results are shown in Table VII. Among them, RSO-SLAM: The algorithm of this paper; W/O-CM: Without Coarse Matching; W/O-MRD: Without Motion Region Detection, this is only the optical flow classification method will not be used for motion region detection, but the normal motion region detection process will still be preserved; W/O-OF: Without Optical Flow Propagation strategy; Only-YL-seg: Only with YOLOv5-seg, this is, directly eliminate all keypoints in the mask with human category.

The experimental results demonstrate that “W/O-CM” has a relatively minor impact when there is slight camera movement. However, in scenarios with significant camera motion, this approach can result in catastrophic failure. “W/O-MRD” shows minimal impact when only human-moving objects are present; however, it becomes considerably affected when various other types of moving objects occupy a larger portion of the scene. In contrast, “W/O-OF” has a more pronounced impact compared to the first two modifications. The absence of the optical flow propagation strategy results in issues such as missed detection compensation and motion continuity problems. This can potentially lead to frames where moving targets

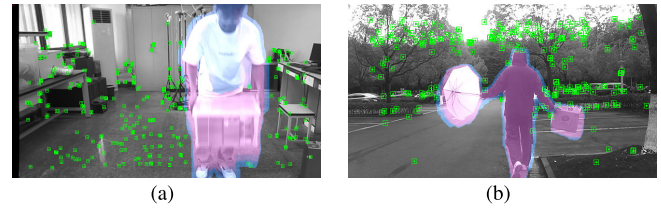


Fig. 14. Real world test: (a) Indoor (b) Outdoor. Regions with deeper shades of red indicate higher motion probability.

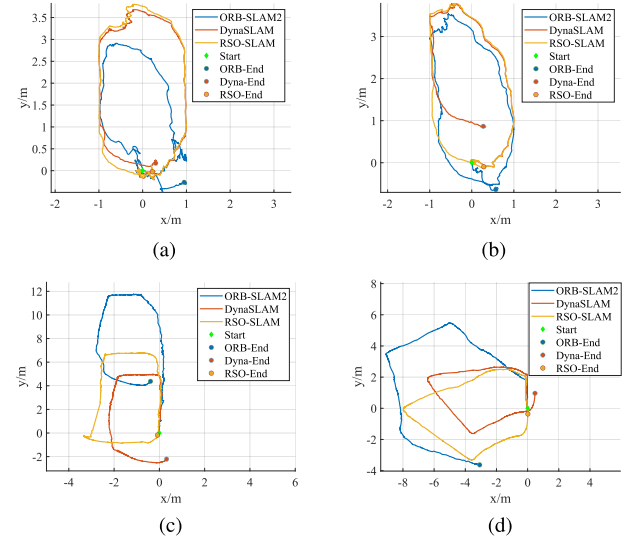


Fig. 15. Real world results: (a) Seq 01, (b) Seq 02, (c) Seq 03, (d) Seq 04.

go undetected. “Only-YL-seg” generally performs worse than other methods due to its relatively simplistic strategy, which results in lost tracking in some sequences.

D. Real World Experiments

To assess the practical efficacy of RSO-SLAM, empirical tests were conducted in both an indoor laboratory and an outdoor parking lot. Utilizing the ZED 2i Stereo Camera, we captured four distinct trajectories, with sequences 01 and 02 documented within the indoor laboratory, and sequences 03 and 04 in the outdoor parking lot, as illustrated in Fig. 14. Within these sequences, an experimenter perpetually moved, introducing additional complexity by randomly moving objects such as boxes and umbrellas. The detailed descriptions of each sequence are presented in the table VIII. The four sets of experiments we recorded were all conducted by the recorder in a circular walk, meaning that the starting and ending points of the recording were at the same position. Therefore, we can measure the positioning accuracy of different methods through end-to-end errors.

The methods compared to RSO-SLAM are ORB-SLAM2 and DynaSLAM, and all of them were tested with closed-loop detection turned off. The experimental results are shown in Fig. 15 and end-to-end error in the table IX. From the experimental results, it can be seen that RSO-SLAM achieved the minimum end-to-end error in all four sets of sequences. ORB-SLAM2 has significant trajectory deviation due to the effective removal of feature points on moving objects due to

TABLE VI
RESULT OF KITTI IN STEREO MODE

Seq	ORB-SLAM2			Stereo-DSO			DynaSLAM			VDO-SLAM			AirDos			RSO-SLAM		
	ATE [m]	RPE [%]	RPE [°/100m]	ATE [m]	RPE [%]	RPE [°/100m]	ATE [m]	RPE [%]	RPE [°/100m]	ATE [m]	RPE [%]	RPE [°/100m]	ATE [m]	RPE [%]	RPE [°/100m]	ATE [m]	RPE [%]	RPE [°/100m]
00	1.30	0.70	0.25	1.35	0.73	0.24	1.4	0.74	0.26	1.35	0.71	0.25	1.34	0.75	0.26	1.27	0.69	0.25
01	10.40	1.39	0.21	10.64	1.48	0.20	9.4	1.57	0.22	10.8	1.11	0.20	9.71	1.42	0.20	9.8	1.46	0.16
02	5.70	0.76	0.23	5.57	0.76	0.23	6.70	0.80	0.24	5.33	0.75	0.22	7.24	0.90	0.33	5.30	0.75	0.22
03	0.60	0.71	0.18	0.85	0.83	0.16	0.60	0.69	0.18	0.55	0.69	0.23	0.88	0.82	0.21	0.56	0.68	0.16
04	0.20	0.48	0.13	0.21	0.48	0.15	0.20	0.45	0.09	0.18	0.44	0.13	0.19	0.45	0.20	0.18	0.45	0.09
05	0.80	0.40	0.16	0.86	0.45	0.16	0.80	0.40	0.16	0.75	0.40	0.16	0.73	0.41	0.17	0.75	0.39	0.15
06	0.80	0.51	0.15	0.78	0.50	0.17	0.80	0.50	0.17	0.96	0.59	0.18	0.62	0.46	0.18	0.74	0.48	0.14
07	0.50	0.50	0.28	0.50	0.52	0.28	0.50	0.52	0.29	0.50	0.49	0.29	0.47	0.48	0.31	0.50	0.48	0.28
08	3.60	1.05	0.32	3.55	1.07	0.32	3.50	1.05	0.32	3.59	1.07	0.33	3.35	0.94	0.32	3.49	1.03	0.31
09	3.20	0.87	0.27	3.31	0.91	0.25	1.60	0.93	0.29	3.74	0.96	0.26	3.54	0.92	0.30	1.70	0.87	0.23
10	1.00	0.60	0.27	1.07	0.60	0.23	1.20	0.67	0.32	1.05	0.61	0.28	1.49	0.69	0.39	1.16	0.64	0.29

TABLE VII
ABLATION EXPERIMENT

Sequence	RSO-SLAM	W/O-CM	W/O-MRD	W/O-OPF	Only-YL-seg
fr3/s/xyz	0.0088	0.0136	0.0095	0.0092	0.0120
fr3/s/half	0.0149	0.0161	0.0127	0.0148	0.0159
fr3/s/static	0.0062	0.0062	0.0068	0.0071	0.0066
fr3/s/rypy	0.0189	0.0206	0.0212	0.0208	0.0292
fr3/w/xyz	0.0146	0.0157	0.0492 *	0.1938 *	0.0161
fr3/w/half	0.0234	0.0231	0.0255	0.0246	0.0251
fr3/w/static	0.0059	0.0075	0.0075	0.0190	0.0161
fr3/w/rypy	0.0330	Failed *	0.0337	0.1633 *	0.0496
balloon	0.0248	0.0266	0.0258	0.0284	0.0275
balloon_tracking	0.0301	0.0322	0.0303	0.0305	0.0329
moving_no_box	0.0201	0.0277	0.0195	0.4176 *	0.4091 *
person_tracking	0.0366	0.0469	0.0445	0.0451	0.0477
placing_no_box	0.0155	0.0220	0.6229 *	0.6158 *	0.7829 *
crowd	0.0166	0.1688 *	0.0193	0.0733 *	0.0296
removing_no_box	0.0153	0.0150	0.0155	0.0151	0.0170
kidnapping_box	0.0251	0.0262	0.0267	0.0270	0.0291

* The * in this table indicates that the method has experienced significant trajectory deviation on this sequence.

TABLE VIII
REAL-WORLD EXPERIMENTS DESCRIPTIONS

Seq	Environment	Duration (s)	Dynamic Objects	Actions
01	Indoor Laboratory	113.1	Person, Box, Keyboard	Sit, Walk, Crouch
02	Indoor Laboratory	87.7	Person, Box, Keyboard	Walk, Run, Jump
03	Outdoor Parking Lot	116.7	Person, Car, Box, Umbrella	Walk, Swing, Spin
04	Outdoor Parking Lot	118.6	Person, Car, Box, Umbrella	Walk, Run, Swing

TABLE IX
REAL-WORLD END-TO-END ERROR [M]

Sequence	ORB-SLAM2	DynaSLAM	RSO-SLAM
01	0.9851	0.3427	0.2260
02	0.8537	0.9088	0.2989
03	4.3972	2.2328	0.1922
04	4.7586	1.0733	0.3606

errors. DynaSLAM found in our experiment that it cannot effectively detect moving objects other than humans, resulting in a certain deviation in the camera trajectory. RSO-SLAM can effectively detect moving objects in most cases of sequences, thus achieving optimal experimental results. Based on the outcomes across diverse datasets, RSO-SLAM showcases effective operation in both indoor and outdoor environments, addressing a variety of dynamic scenarios. In comparison to numerous current cutting-edge methods limited to specific scenarios, RSO-SLAM excels in adapting to diverse and intricate environments, underscoring its heightened robustness.

E. Algorithm Computation Time

Table X offers a comparative analysis of the time efficiency between RSO-SLAM and other SLAM systems. In contrast to

TABLE X
COMPARISON OF COMPUTATION TIMES ON TUM [MS]

Method	ORB-SLAM2	DS-SLAM	Dyna-SLAM	CFP-SLAM	RSO-SLAM
Each Frame	23.8	63.25	499.39	42.7	45.6

TABLE XI
AVERAGE RUNNING TIME OF EACH MODULE [MS]

Module	LiteFlowNet2	Yolov5-seg	MRD	MPC	Each Frame
TTime Cost(ms)	24.62	12.73	14.08	4.11	45.6

ORB-SLAM2, our method introduces some additional runtime due to incorporating modules designed for dynamic scene processing. However, when compared to traditional DS-SLAM and Dyna-SLAM, our approach significantly reduces system runtime. In comparison to the recent CFP-SLAM, our runtime is comparable, but RSO-SLAM boasts broader applicability across diverse scenarios and achieves higher positioning accuracy. Table XI presents the runtime performance of each module in RSO-SLAM. These modules, including LiteFlowNet2 for optical flow calculation, Yolov5 seg for instance segmentation, and the feature point extraction process of the original ORB-SLAM2, utilize multi-threaded parallel operations. Subsequently, the motion region detection module (MRD) and motion probability calculation module (MPC) operate sequentially following the completion of the preceding processes.

RSO-SLAM capitalizes on CUDA (Compute Unified Device Architecture) parallel computing acceleration, notably improving the efficiency of optical flow computation and instance segmentation. On average, it processes each frame in approximately 46 ms, equivalent to around 22 frames per second (FPS). While our current algorithm's real-time performance may be constrained by hardware limitations and may not meet high-speed requirements, it adequately serves the needs of most practical scenarios. RSO-SLAM mandates a minimum 6GB VRAM requirement. In platforms with constrained computational capabilities, such as embedded systems, real-time performance depends on the operational speed of the optical flow and instance segmentation networks.

VII. CONCLUSION

This paper presents an innovative visual SLAM algorithm that effectively handles the challenges posed by complex

dynamic scenes through the integration of instance segmentation and optical flow techniques. The proposed RSO-SLAM algorithm capitalizes on the KMC methodology and seamlessly incorporates semantic and optical flow information to identify potential motion regions and prominent motion regions within the scene. Additionally, an optical flow attenuation propagation strategy is employed to achieve meticulous motion probability calculation, inter-frame propagation, and missed detection compensation of moving targets. A comprehensive evaluation demonstrates the algorithm's remarkable localization accuracy across diverse complex dynamic scenarios, encompassing high-dynamic settings, low-dynamic environments, non-rigid objects, and unanticipated objects, emphasizing its robustness in challenging conditions.

Moving forward, there are still aspects of RSO-SLAM that can be further enhanced. When the field of view is dominated by moving objects within the scene, the application of a mere feature point filtering method may lead to system failure. Moreover, the effectiveness of our motion regions detection method is notably impacted when the scene undergoes significant changes. For RSO-SLAM deployed on devices with constrained computational resources, inter-module interference may impede the main thread's progress, leading to significantly increased computation time. Given that the architecture of RSO-SLAM is based on ORB-SLAM2, additional optimization is required to extend its applicability to other VSLAM methods. Tackling these challenges opens up promising paths for future research.

REFERENCES

- [1] J. Engel, V. Koltun, and D. Cremers, "Direct sparse odometry," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 40, no. 3, pp. 611–625, Mar. 2018.
- [2] J. Engel, T. Schöps, and D. Cremers, "LSD-SLAM: Large-scale direct monocular SLAM," in *Proc. Eur. Conf. Comput. Vis.*, Zurich, Switzerland, Cham, Switzerland: Springer, Sep. 2014, pp. 834–849.
- [3] R. Mur-Artal and J. D. Tardós, "ORB-SLAM2: An open-source SLAM system for monocular, stereo, and RGB-D cameras," *IEEE Trans. Robot.*, vol. 33, no. 5, pp. 1255–1262, Oct. 2017.
- [4] D.-H. Kim and J.-H. Kim, "Effective background model-based RGB-D dense visual odometry in a dynamic environment," *IEEE Trans. Robot.*, vol. 32, no. 6, pp. 1565–1573, Dec. 2016.
- [5] S. Li and D. Lee, "RGB-D SLAM in dynamic environments using static point weighting," *IEEE Robot. Autom. Lett.*, vol. 2, no. 4, pp. 2263–2270, Oct. 2017.
- [6] Y. Sun, M. Liu, and M. Q.-H. Meng, "Improving RGB-D SLAM in dynamic environments: A motion removal approach," *Robot. Auto. Syst.*, vol. 89, pp. 110–122, Mar. 2017.
- [7] Y. Sun, M. Liu, and M. Q.-H. Meng, "Motion removal for reliable RGB-D SLAM in dynamic environments," *Robot. Auto. Syst.*, vol. 108, pp. 115–128, Oct. 2018.
- [8] R. Scona, M. Jaimez, Y. R. Petillot, M. Fallon, and D. Cremers, "StaticFusion: Background reconstruction for dense RGB-D SLAM in dynamic environments," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2018, pp. 3849–3856.
- [9] W. Dai, Y. Zhang, P. Li, Z. Fang, and S. Scherer, "RGB-D SLAM in dynamic environments using point correlations," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 1, pp. 373–389, Jan. 2022.
- [10] Q. U. Islam, H. Ibrahim, P. K. Chin, K. Lim, and M. Z. Abdullah, "MVS-SLAM: Enhanced multiview geometry for improved semantic RGBD SLAM in dynamic environment," *J. Field Robot.*, vol. 41, no. 1, pp. 109–130, Jan. 2024.
- [11] F. Zhong, S. Wang, Z. Zhang, C. Chen, and Y. Wang, "Detect-SLAM: Making object detection and SLAM mutually beneficial," in *Proc. IEEE Winter Conf. Appl. Comput. Vis. (WACV)*, Mar. 2018, pp. 1001–1010.
- [12] J. Vincent, M. Labbé, J.-S. Lauzon, F. Grondin, P.-M. Comtois-Rivet, and F. Michaud, "Dynamic object tracking and masking for visual SLAM," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2020, pp. 4974–4979.
- [13] A. Li, J. Wang, M. Xu, and Z. Chen, "DP-SLAM: A visual SLAM with moving probability towards dynamic environments," *Inf. Sci.*, vol. 556, pp. 128–142, May 2021.
- [14] T. Ji, C. Wang, and L. Xie, "Towards real-time semantic RGB-D SLAM in dynamic environments," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2021, pp. 11175–11181.
- [15] N. Dvornik, K. Shmelkov, J. Mairal, and C. Schmid, "BlitzNet: A real-time deep network for scene understanding," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 4154–4162.
- [16] X. Hu et al., "CFP-SLAM: A real-time visual SLAM based on coarse-to-fine probability in dynamic environments," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2022, pp. 4399–4406.
- [17] L. Yan, X. Hu, L. Zhao, Y. Chen, P. Wei, and H. Xie, "DGS-SLAM: A fast and robust RGBD SLAM in dynamic environments combined by geometric and semantic information," *Remote Sens.*, vol. 14, no. 3, p. 795, Feb. 2022.
- [18] G. Li et al., "RGBD-SLAM based on object detection with two-stream YOLOv4-MobileNetv3 in autonomous driving," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 3, pp. 2847–2857, Jun. 2023.
- [19] D. Kong, X. Li, Q. Xu, Y. Hu, and P. Ni, "SC_LPR: Semantically consistent LiDAR place recognition based on chained cascade network in long-term dynamic environments," *IEEE Trans. Image Process.*, vol. 33, pp. 2145–2157, 2024.
- [20] J. Zhang, M. Henein, R. Mahony, and V. Ila, "VDO-SLAM: A visual dynamic object-aware SLAM system," 2020, *arXiv:2005.11052*.
- [21] T. Zhang, H. Zhang, Y. Li, Y. Nakamura, and L. Zhang, "FlowFusion: Dynamic dense RGB-D SLAM based on optical flow," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2020, pp. 7322–7328.
- [22] J. Cheng, Z. Wang, H. Zhou, L. Li, and J. Yao, "DM-SLAM: A feature-based SLAM system for rigid dynamic scenes," *ISPRS Int. J. Geo-Inf.*, vol. 9, no. 4, p. 202, Mar. 2020.
- [23] D. Esparza and G. Flores, "The STDyn-SLAM: A stereo vision and semantic segmentation approach for VSLAM in dynamic outdoor environments," *IEEE Access*, vol. 10, pp. 18201–18209, 2022.
- [24] W. Chen et al., "A monocular-visual SLAM system with semantic and optical-flow fusion for indoor dynamic environments," *Micromachines*, vol. 13, no. 11, p. 2006, Nov. 2022.
- [25] Y. Fang, Z. Xie, K. Chen, G. Huang, R. Zarei, and Y. Xie, "DYS-SLAM: A real-time RGBD SLAM combined with optical flow and semantic information in a dynamic environment," *J. Intell. Fuzzy Syst.*, vol. 45, no. 6, pp. 10349–10367, 2023.
- [26] C. Yu et al., "DS-SLAM: A semantic visual SLAM towards dynamic environments," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2018, pp. 1168–1174.
- [27] B. Bescos, J. M. Fàcil, J. Civera, and J. Neira, "DynaSLAM: Tracking, mapping, and inpainting in dynamic scenes," *IEEE Robot. Autom. Lett.*, vol. 3, no. 4, pp. 4076–4083, Oct. 2018.
- [28] G. Jocher et al., *Ultralytics/YOLOv5: V7. 0-YOLOv5 Sota Realtime Instance Segmentation*. Genève, Switzerland: Zenodo, 2022.
- [29] T.-W. Hui, X. Tang, and C. C. Loy, "A lightweight optical flow CNN—Revisiting data fidelity and regularization," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 43, no. 8, pp. 2555–2569, Aug. 2021.
- [30] J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers, "A benchmark for the evaluation of RGB-D SLAM systems," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, Vilamoura-Algarve, Portugal, Oct. 2012, pp. 573–580.
- [31] E. Palazzolo, J. Behley, P. Lottes, P. Giguere, and C. Stachniss, (2019). *ReFusion: 3D Reconstruction in Dynamic Environments for RGB-D Cameras Exploiting Residuals*. [Online]. Available: <https://www.ipb.uni-bonn.de/pdfs/palazzolo2019iros.pdf>
- [32] A. Geiger, P. Lenz, C. Stiller, and R. Urtasun, "Vision meets robotics: the KITTI dataset," *Int. J. Robot. Res.*, vol. 32, no. 11, pp. 1231–1237, Sep. 2013.
- [33] V. Badrinarayanan, A. Kendall, and R. Cipolla, "SegNet: A deep convolutional encoder-decoder architecture for image segmentation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 12, pp. 2481–2495, Dec. 2017.
- [34] K. He, G. Gkioxari, P. Dollár, and R. Girshick, "Mask R-CNN," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 2961–2969.

- [35] Y. Fan, Q. Zhang, Y. Tang, S. Liu, and H. Han, "Blitz-SLAM: A semantic SLAM in dynamic environments," *Pattern Recognit.*, vol. 121, Jan. 2022, Art. no. 108225.
- [36] T. Qin, P. Li, and S. Shen, "VINS-mono: A robust and versatile monocular visual-inertial state estimator," *IEEE Trans. Robot.*, vol. 34, no. 4, pp. 1004–1020, Aug. 2018.
- [37] G. Hamerly and C. Elkan, "Learning the k in k-means," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 16, 2003, pp. 281–288.
- [38] E. Palazzolo, J. Behley, P. Lottes, P. Giguère, and C. Stachniss, "ReFusion: 3D reconstruction in dynamic environments for RGB-D cameras exploiting residuals," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Nov. 2019, pp. 7855–7862.
- [39] R. Wang, M. Schworer, and D. Cremers, "Stereo DSO: Large-scale direct sparse visual odometry with stereo cameras," in *Proc. IEEE Int. Conf. Comput. Vis.*, Oct. 2017, pp. 3903–3911.



Zhenyu Chen received the bachelor's degree in the Internet of Things from the University of Electronic Science and Technology of China, Chengdu, China, in 2023. His research interests include SLAM.



Xiaotong Kong received the bachelor's degree in communication engineering from the University of Electronic Science and Technology of China, in 2021, where he is currently pursuing the master's degree in electronic information. His main research interests include indoor positioning methods.



Liang Qin received the bachelor's degree in the Internet of Things from the University of Electronic Science and Technology of China, Chengdu, China, in 2023. His research interests include SLAM and embodied artificial intelligence.



Zejie Lv received the bachelor's degree in electronic information engineering from the University of Electronic Science and Technology of China, Chengdu, China, in 2022. He is currently pursuing the master's degree in electronic information engineering. His current research interests include lidar and multi-sensor fusion positioning and navigation.



Chang Wu (Senior Member, IEEE) received the Ph.D. degree from the University of Electronic Science and Technology of China (UESTC), in 2008. He is currently an Associate Professor with the School of Information and Communication, UESTC. His research interests include indoor location-based service, navigation, communication theory, information technology, channel coding, 5G mobile communication systems, and artificial intelligence.



Zhiqi Zhao received the bachelor's degree in communication engineering from the University of Electronic Science and Technology of China, in 2022, where he is currently pursuing the master's degree in electronic information. His main research interests include indoor positioning methods.